Shelf Voice — 07 Data Model and Integrations

Oct 27, 2026 · @Aradhya Chaudhary

Status and open decisions

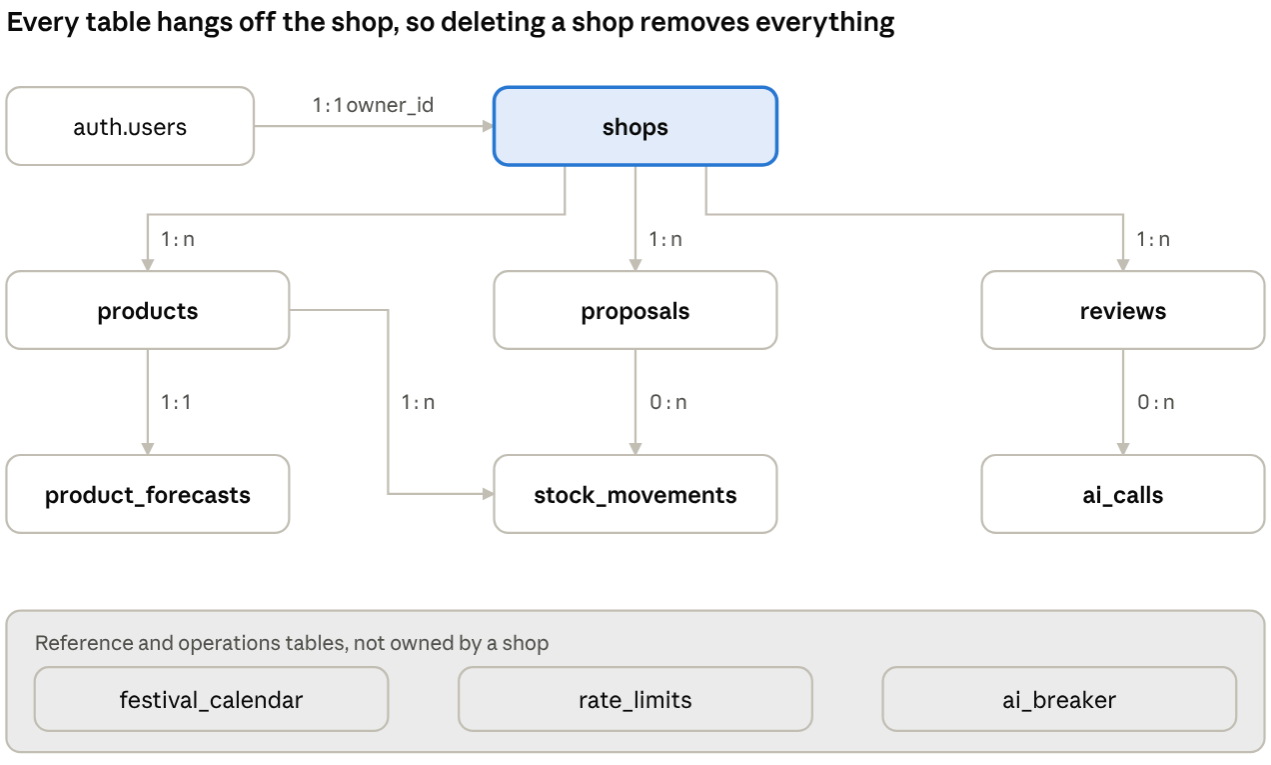
Version 1.0, approved by the owner on Oct 27. All six data-model decisions are final and reflected in the tables below; changes from here go through this table.

#	Decision	Options (recommendation %)	Why the top option	Owner's choice
K1	Primary keys	UUID v4 via `gen_random_uuid()` (65%); UUID v7 from the app (20%); bigint identity (15%)	Built into Postgres 17, unguessable in URLs, no extension; time order comes from `created_at`	UUID v4, gen_random_uuid()
K2	How stock is stored	Movement log + current stock kept on the product by a trigger (70%); movement log only, stock computed by a view (20%); a single counter, no log (10%)	Full history for forecasts and audits, plus a fast stock read for lists	Movement log + trigger-kept stock
K3	Money	Integer paise (75%); `numeric(10,2)` rupees (25%)	Exact maths, no rounding drift; formatted to Rs only for display	Integer paise
K4	Product names	Display name + aliases list (Hindi, spoken variants) (65%); one name only (35%)	Better matching of "Maggi", "maggie", "मैगी" and stronger transcription keyword hints	Name + aliases list

#	Decision	Options (recommendation %)	Why the top option	Owner's choice
K5	Storefront address (slug)	From shop name + 4 random characters, editable later (65%); owner types one at setup (35%)	Zero extra step during voice setup; no collisions	Auto from shop name, editable
K6	Festival calendar	Hand-curated table of major 2026-27 festivals, seeded by migration (75%); public holiday API (25%)	15 rows, no external dependency, festivals that move stock (Diwali, Holi, Eid, Navratri) chosen deliberately	Hand-curated seeded table

Entity relationships

The shop is the root of all owner data: one Google account owns one shop, and every other owner table links back to it with `on delete cascade`.



entity relationships · 7 shop tables, 3 reference tables

A confirmed proposal creates zero or more stock movements. ai_calls also carries shop_id and, where relevant, a proposal or review ID; only the review link is drawn to keep the picture readable.

Tables and columns

Ten tables in the `public` schema; every row that belongs to a shop carries `shop_id`, and every table has RLS enabled. Types assume the recommended options (UUID v4 keys, money in paise).

Table	Columns (type, constraint)	Notes
shops	id uuid pk; owner_id uuid fk auth.users, unique; name text 1-80; slug text unique, `^[a-z0-9-]{3,48}$`; category text; phone text E.164 or null; address text; timezone text default Asia/Kolkata; hours jsonb (per weekday open and close); language text in (hi, en); is_published bool default false; is_demo bool; demo_expires_at timestamptz; lead_time_days int 1-30 default 2; review_period_days int 1-30 default 7; created_at, updated_at timestamptz	One shop per owner. Demo copies expire after 24 h
products	id uuid pk; shop_id fk shops on delete cascade; name text 1-80; aliases text[] max 10; category text; unit text; price_paise int > 0 or null; stock_qty int >= 0 or null; is_in_stock bool; low_stock_level int default 5; pack_size int default 1; image_path text; is_published bool default true; archived_at timestamptz; created_at, updated_at	Null price = not spoken yet. Null stock = owner does not track counts for this item

Table	Columns (type, constraint)	Notes
stock_movements	id uuid pk; shop_id fk; product_id fk products; kind enum SALE, DELIVERY, CORRECTION; qty int (SALE and DELIVERY > 0; CORRECTION any non-zero); unit_price_paise int; occurred_at timestamptz; source enum VOICE, BILL, TAP, SEED; proposal_id fk proposals or null; idempotency_key text; created_at	Append-only. Unique (shop_id, idempotency_key)
proposals	id uuid pk; shop_id fk; kind enum SETUP, COMMAND, BILL, SHELF; status enum PENDING, CONFIRMED, CANCELLED, FAILED, EXPIRED; input_path text; transcript text max 4000; changes jsonb (validated list); confirmed_changes jsonb; created_at; decided_at	Pending proposals expire after 24 h
reviews	id uuid pk; shop_id fk; audio_path text; audio_deleted_at timestamptz; transcript text max 1000; status enum PENDING, ANALYZED, FAILED; sentiment enum POSITIVE, NEUTRAL, NEGATIVE; summary text max 200; mentioned_product_ids uuid[]; stockout_product_ids uuid[]; is_hidden bool; consent_at timestamptz not null; ip_hash text; created_at	No customer identity stored
product_forecasts	product_id uuid pk fk; shop_id fk; level numeric; sigma numeric; days_left numeric; reorder_point int; order_qty int; stockout_boost_until date; explanation text; explanation_lang text; explanation_on date; computed_at	One row per product, rewritten on each refresh

Table	Columns (type, constraint)	Notes
ai_calls	id uuid pk; request_id text; shop_id fk or null; proposal_id or review_id; job enum; model text; prompt_version text; input_tokens, cached_tokens, output_tokens, audio_seconds, images int; cost_usd numeric(10,6); latency_ms int; retries int; outcome enum; owner_action enum or null; created_at	Shape from document 05; 90-day retention
festival_calendar	id smallint pk; name text; starts_on date; ends_on date; factor numeric 1.0-3.0; categories text[]	Read-only reference data (K6)
rate_limits	key text; window_start timestamptz; count int; primary key (key, window_start)	Written only by a server function (P4)
ai_breaker	provider text pk; state enum CLOSED, OPEN, HALF_OPEN; failures int; opened_at timestamptz	Shared circuit-breaker state (document 05)

The team admin allow-list lives in an environment variable, not a table.

Indexes, derived values and RLS

Every query the app makes is served by an index that starts with `shop_id`, which is also the column every RLS policy filters on.

Index	Serves
shops (owner_id) unique	RLS lookups, one shop per owner
shops (slug) unique	Storefront and review page routing
products (shop_id, category, name)	Stock tab and storefront lists
products using gin (lower(name) gin_trgm_ops) and gin (aliases)	Fuzzy matching of spoken names to products
stock_movements (shop_id, product_id, occurred_at desc)	Forecast refresh, sales history

Index	Serves
stock_movements (shop_id, idempotency_key) unique	Duplicate protection
proposals (shop_id, status, created_at desc)	Pending proposals, expiry job
reviews (shop_id, created_at desc)	Review feed
reviews (status) where status = 'PENDING'	Nightly retry of unanalysed reviews
ai_calls (created_at), ai_calls (shop_id, created_at)	Budget view, admin dashboard

Derived values (K2): an `after insert` trigger on stock_movements updates `products.stock_qty` in the same transaction (SALE subtracts, DELIVERY adds, CORRECTION applies its signed amount). If the result would go below zero, the insert fails and the app asks the owner to correct the count. A nightly check recomputes stock from the full movement log and logs any difference.

Views:

- `public_storefront` (security_invoker): published shops joined to their published, non-archived products, exposing name, unit, price and a boolean `in_stock` only.
- `ai_spend_daily`: cost per day and per job from ai_calls, for the budget alerts in document 05.

RLS summary (full rules in document 03):

Table	Owner	Public	Server (service role)
shops, products, product_forecasts	Read and write own	Through public_storefront only	All
stock_movements, proposals	Read and insert own; no update or delete (stock corrections are new rows). Proposals: owner may update status only while PENDING	None	All
reviews	Read own; update is_hidden only	None	Insert, analysis updates

Table	Owner	Public	Server (service role)
ai_calls	Read own	None	Insert
festival_calendar	Read	Read	All
rate_limits, ai_breaker	None	None	All

Data lifecycle

Every kind of data has an owner, a place, and a fixed end; the daily cleanup job enforces the dates in this table.

Data	Where	Created by	Kept	Deleted by
Owner voice notes	Storage `voice/{shop_id}/owner/{proposal_id}.wav`	Owner app upload	7 days	Daily job
Customer review audio	Storage `voice/{shop_id}/reviews/{review_id}.wav`	Review page upload	30 days (P1)	Daily job; sets `audio_deleted_at`
Shelf and bill photos	Storage `photos/{shop_id}/{proposal_id}.jpg`	Owner app upload	30 days	Daily job
Product cut-out images	Storage `catalog-images/{shop_id}/{product_id}.webp`	Shelf proposal confirm	While the product exists	Product delete or shop delete
Proposals	proposals table	Server	Pending 24 h, then EXPIRED; kept 90 days	Daily job
Stock movements, products, forecasts	Tables	Owner actions, server	Until shop deletion	Shop delete
Reviews (text and analysis)	reviews table	Server	Until shop deletion, or hidden by owner	Shop delete
AI call metadata	ai_calls table	Server	90 days	Daily job
Rate-limit counters	rate_limits table	Server	2 days	Daily job
Demo shop copies	All tables and storage	Demo sign-in	24 hours	Daily job

Shop deletion (FR-16): one server function deletes the shop row (cascading to every child table), removes the shop's storage folders, then deletes the auth user; it finishes within 24 hours, with a retry if storage removal fails. Unconfirmed uploads that never became a proposal are swept after 24 hours.

Migration strategy

The schema lives only in version-controlled SQL migrations; nobody changes a database through the dashboard.

- Tooling: Supabase CLI. `supabase migration new <name>` creates a timestamped file in `supabase/migrations/`; `supabase db reset` rebuilds the local database from all migrations plus `seed.sql`; `supabase db push` applies new migrations to staging, then production.
- Order: preview branch runs against staging; production gets the same migrations only after the release checklist (document 08).
- Each migration enables RLS and creates policies in the same file as the table it adds, so a table is never live without them; the CI check in document 04 enforces this.
- Additive first: new columns are nullable or have defaults; a column is removed only in a later migration after code stops using it.
- Types: `supabase gen types typescript` regenerates `src/types/database.ts` after every migration, and CI fails if the committed file is stale.
- Postgres version: Supabase's platform has moved to Postgres 17. Shelf Voice needs only `pgcrypto` (for `gen_random_uuid`), `pg_trgm` (fuzzy name matching) and `pg_cron` (scheduled jobs); their availability is checked on day 1 in the project's extensions list. Extensions dropped from Supabase's Postgres 17 (timescaledb, plv8, pgjwt) are not used.

Planned first migrations:

1. Extensions and enums.
2. shops and products, with RLS and indexes.
3. stock_movements with the stock trigger, proposals.
4. reviews, product_forecasts, ai_calls.
5. festival_calendar seed, rate_limits, ai_breaker.
6. Views (`public_storefront`, `ai_spend_daily`), storage buckets and storage policies.
7. pg_cron schedules (forecast refresh, cleanup).

Meta Model API integration

All three models sit behind one server module and one API key; Meta's limits are far above what an 8-day demo needs, so the real constraint is the $150 credit.

Model	Endpoint	Request	Price (Meta pricing page)	Limits (per team)
muse-spark-1.3 (Standard)	POST https://api.meta.ai/v1/responses	Developer message + user input (text or image); JSON schema via `text.format`, or `tools` for commands; low reasoning effort	$1.25 in, $0.15 cached in, $4.25 out per 1M tokens	3,000 requests and 4M tokens per minute
muse-voice-transcribe-1.0	POST https://api.meta.ai/v1/asr/transcribe	Mono 16-bit PCM WAV at 16 or 24 kHz; `mode` PUSH_TO_TALK; `keywords` (shop's product names and aliases); `languageBias` (Hindi, English)	$0.18 per audio hour, billed in whole seconds	No per-minute limit; 256 concurrent and 16,000 started per hour
sam-3.1	POST https://api.meta.ai/v1/responses with model sam-3.1	Shelf photo + short text prompt ("packaged product")	$2.50 per 1,000 images	3,000 requests per minute

Authentication: bearer API key from the server's environment only. Standard models do not use our content for training; the contributor variants are never called.

Failure contract (shared by all three):

Response	Our handling
2xx with valid output	Parse with Zod; store; log `ai_calls` OK
2xx with invalid or empty output	No retry; SCHEMA_REJECTED; owner sees "couldn't understand" with the clip kept
429	One retry after 1-3 s with jitter; then RATE_LIMITED
5xx or timeout (30 s Spark, 20 s transcription and SAM)	One retry; then ERROR; breaker counts it
401 or 403	No retry; critical alert (key revoked or region blocked)

Still to confirm on day 1, because public docs do not state them: the exact multipart field names and response shape of `/v1/asr/transcribe`, and the mask format SAM returns. A 10-minute spike with the hackathon key settles both before any feature code.

Other integrations

Every other integration is either a link, a webhook or a hosted SDK; none needs a separate backend.

Integration	Purpose	How it connects	Secret and where	If it fails
Google sign-in via Supabase Auth	Owner sign-in (T4)	OAuth client in Google Cloud; redirect URI is the Supabase project's auth callback; scopes openid, email, profile only	Client secret in the Supabase dashboard	Owners can still use "Try the demo shop"; sign-in error page explains
Cloudflare Turnstile	Bot check on review submit (P2)	Invisible widget on P-2 returns a token; server verifies it with Cloudflare's siteverify endpoint	Secret key in Vercel env; site key public	Review rejected with a retry message; rate limits still apply

Integration	Purpose	How it connects	Secret and where	If it fails
WhatsApp click-to-chat	Customer orders and asks (PRD D4)	A `https://wa.me/<number>?text=<encoded message>` link built on the storefront; number in international format without `+`	None	Call button remains
Axiom	Logs, errors, Web Vitals, monitors (O1)	next-axiom library from the app	Ingest-only token in Vercel env	Logs lost for the outage; app unaffected
Better Stack	Uptime, heartbeats, status page (O3)	Outside checks of `/api/health*`; cron jobs ping a heartbeat URL	Heartbeat URL in Vercel env	No outside alerting; Axiom monitors still alert
Discord webhook	Alert channel (O4)	Axiom monitors, Better Stack and the budget job post to one channel webhook	Webhook URL in Vercel env and tool settings	Alerts also go to email
Vercel cron	Daily cleanup, keep-alive, backup trigger	`vercel.json` cron entries calling protected routes	`CRON_SECRET` bearer header	Missed run caught by the Better Stack heartbeat
pg_cron	Nightly forecast refresh, proposal expiry	Scheduled SQL functions inside Postgres	None	Next run recomputes everything

The Google OAuth consent screen must be set to production, not testing, before real owners sign in, because testing mode allows only listed test users. Only basic scopes are requested.

Idempotency, retries and consistency

Every write that can be repeated by a retry, a double tap or a flaky connection carries a key that makes the second attempt a no-op.

Operation	Idempotency key	Consistency rule
Upload clip or photo	Client-generated UUID in the storage path	Re-upload overwrites the same object
Create proposal from a clip	Storage path of the clip (unique per proposal)	Same clip returns the existing proposal instead of calling Meta again
Confirm proposal	Proposal ID	One transaction: inserts movements and product changes, sets status CONFIRMED; a second confirm returns the stored result
Tap-to-log a sale	Client UUID per tap	Unique (shop_id, idempotency_key) on stock_movements
Submit review	Client UUID created when the page opens	Unique per review row
Forecast refresh	None needed	Recomputed from the full movement log each time
Shop delete	Shop ID	Safe to rerun; skips what is already gone

Offline queue: the browser keeps unsent clips with their keys in IndexedDB and retries with backoff; because the keys are stable, a clip sent twice still creates one proposal.

Ordering: a proposal is checked against the catalog at confirm time, not at proposal time. If a product it changes was edited or archived in between, that change card is marked "Please check" instead of being applied blindly.

Costs, limits and lock-in

The only paid usage during the hackathon is Meta API calls, covered by the $150 credit; every other provider stays on its free plan.

Provider	Cost through Nov 9	Limit that could bite	Lock-in and way out
Meta Model API	Estimated under $20 of the $150 credit	Credit balance (alerts at 50% and 80%)	Low: one module, OpenAI-compatible; OpenRouter also serves Muse Spark and Voice Transcribe
Supabase (free)	$0	500 MB database; project pauses after 7 days idle (keep-alive covers it)	Low: plain Postgres dump; Storage files downloadable; auth users exportable
Vercel Hobby	$0	Non-commercial use only; 1 hour of logs	Low: standard Next.js runs on any Node host
Cloudflare Turnstile	$0	20 widgets per account	Low: swap for another challenge provider in one component
Axiom (free)	$0	30-day retention	Low: structured JSON logs; change the transport
Better Stack (free)	$0	10 monitors	Low: health endpoints are tool-agnostic
Google OAuth	$0	Consent screen must be in production for real owners	Medium: owners' accounts are tied to Google identities; adding phone sign-in later keeps the same user table

After the hackathon, commercial use needs Vercel Pro ($20 per member per month) and Supabase Pro ($25 per month) at minimum; a full cost model belongs in the post-hackathon plan, not this MVP.

Integration testing

Each integration has one contract test against recorded responses on every PR and one live smoke test before release; the full plan is in document 04.

Integration	On every PR (recorded)	Before release (live)
Muse Spark	Schema parsing, tool-call validation, each failure response in the contract table	Eval suite; one setup, command, bill and review end to end
Voice Transcribe	Request built with correct format, keywords and language hints; response parsing	5 fixture clips; product-name accuracy reported
SAM 3.1	Mask decoding and cut-out cropping from a recorded response	2 shelf photos
Supabase	Real local stack: RLS, trigger, views, migrations from zero	Migrations applied to staging, then production
Google sign-in	Mocked session in E2E	One real sign-in on staging and on production
Turnstile	Cloudflare's published test keys (always pass, always fail)	Real widget on staging
WhatsApp link	Unit test of URL encoding with Hindi text and special characters	Opened on a real phone
Axiom, Better Stack, Discord	Logger redaction unit tests	One forced error and one forced downtime produce an alert in Discord
Cron jobs	Cleanup and shop-delete functions run against the local stack with frozen time	Heartbeat seen in Better Stack after first run

Sources

Checked Oct 27, 2026.

- Meta Model API pricing and rate limits: prices, per-team limits, standard vs contributor data use for Muse Spark 1.3, Voice Transcribe and SAM 3.1
- Build with Muse Voice Transcribe on Meta Model API: file endpoint, audio format, keywords and languageBias parameters
- Meta Model Cookbook: voice API fundamentals: file and realtime transcription scripts
- SAM 3.1 demo client: sam-3.1 called through the Responses API

- Supabase Postgres 17 release notes: platform move to Postgres 17 and extensions dropped from it
- WhatsApp click-to-chat links explained: wa.me link and prefilled text format (third-party)